```
import asyncio
import hashlib
import logging
import os
import re
import tempfile
from pathlib import Path

import yt_dlp
from aiogram import Bot, Dispatcher, F
from aiogram.filters import CommandStart
from aiogram.types import (
    FSInputFile,
    InlineQuery,
    InlineQueryResultArticle,
    InlineQueryResultCachedVideo,
    InputTextMessageContent,
    Message,
)

TOKEN = os.getenv("BOT_TOKEN")
PROXY = os.getenv("PROXY")  # например socks5://127.0.0.1:1080, если TikTok не от
DELETE_ORIGINAL = os.getenv("DELETE_ORIGINAL", "0") == "1"
# ID чата, куда бот заливает видос, чтобы получить file_id (нужно для inline-режи
STORAGE_CHAT_ID = os.getenv("STORAGE_CHAT_ID")
INLINE_WAIT = 8  # сколько секунд ждём видос, пока Telegram не протух запрос
MAX_SIZE = 50 * 1024 * 1024

TT_RE = re.compile(r"https?://(?:www\.|vm\.|vt\.|m\.)?tiktok\.com/\S+", re.I)

logging.basicConfig(level=logging.INFO)
dp = Dispatcher()

file_cache: dict[str, str] = {}          # ссылка -> file_id
tasks: dict[str, asyncio.Task] = {}      # ссылка -> задача, которая сейчас качае


def download(url: str, folder: str):
    opts = {
        "outtmpl": str(Path(folder) / "%(id)s.%(ext)s"),
        "format": "best[ext=mp4]/best",
        "noplaylist": True,
        "quiet": True,
```

```
        "max_filesize": MAX_SIZE,
    }
    if PROXY:
        opts["proxy"] = PROXY
    with yt_dlp.YoutubeDL(opts) as ydl:
        info = ydl.extract_info(url, download=True)
        return ydl.prepare_filename(info), info


def clean(url: str) -> str:
    return url.rstrip(".,)!?")


# ---------- обычный режим: ссылка в чате -> видос ----------

@dp.message(CommandStart())
async def start(m: Message):
    await m.answer(
        "Кидай ссылку на TikTok, верну видосом.\n"
        "Или в любом чате пиши @имя_бота и вставь ссылку."
    )


@dp.message(F.text | F.caption)
async def handle(m: Message, bot: Bot):
    text = m.text or m.caption or ""
    links = [clean(u) for u in TT_RE.findall(text)][:3]
    if not links:
        return

    sent_any = False
    for url in links:
        try:
            await bot.send_chat_action(m.chat.id, "upload_video")
            with tempfile.TemporaryDirectory() as tmp:
                path, info = await asyncio.to_thread(download, url, tmp)
                if os.path.getsize(path) > MAX_SIZE:
                    await m.reply("Видос больше 50 МБ, Telegram не даст отправить
                    continue
                caption = f"От {m.from_user.full_name}" if DELETE_ORIGINAL and m.
                await m.reply_video(
                    FSInputFile(path),
                    caption=caption,
                    width=info.get("width"),
                    height=info.get("height"),
                    duration=int(info["duration"]) if info.get("duration") else N
```

```
                    supports_streaming=True,
                )
                sent_any = True
        except Exception as e:
            logging.exception("download failed: %s", e)
            await m.reply("Не смог скачать. Может, это фотопост, видео закрытое и

    if sent_any and DELETE_ORIGINAL:
        try:
            await m.delete()
        except Exception:
            pass


# ---------- inline-режим: @бот ссылка -> видос в меню над чатом ----------

async def fetch_file_id(bot: Bot, url: str) -> str:
    """Качает видос, заливает в хранилище, возвращает file_id."""
    with tempfile.TemporaryDirectory() as tmp:
        path, info = await asyncio.to_thread(download, url, tmp)
        if os.path.getsize(path) > MAX_SIZE:
            raise RuntimeError("больше 50 МБ")
        msg = await bot.send_video(
            int(STORAGE_CHAT_ID),
            FSInputFile(path),
            width=info.get("width"),
            height=info.get("height"),
            duration=int(info["duration"]) if info.get("duration") else None,
            supports_streaming=True,
        )
    file_cache[url] = msg.video.file_id
    return msg.video.file_id


async def get_file_id(bot: Bot, url: str) -> str:
    if url in file_cache:
        return file_cache[url]
    task = tasks.get(url)
    if not task:
        task = asyncio.create_task(fetch_file_id(bot, url))
        tasks[url] = task

        def _done(t: asyncio.Task):
            tasks.pop(url, None)
            if not t.cancelled():
                t.exception()  # чтобы не ругался на неполученную ошибку
```

```
        task.add_done_callback(_done)
    # shield: если ждать надоело, загрузка всё равно продолжится в фоне
    return await asyncio.wait_for(asyncio.shield(task), timeout=INLINE_WAIT)


def article(rid: str, title: str, desc: str, text: str) -> InlineQueryResultArtic
    return InlineQueryResultArticle(
        id=rid,
        title=title,
        description=desc,
        input_message_content=InputTextMessageContent(message_text=text),
    )


@dp.inline_query()
async def inline_handler(q: InlineQuery, bot: Bot):
    match = TT_RE.search(q.query or "")
    if not match:
        await q.answer(
            [article("hint", "Вставь ссылку на TikTok", "@бот https://vm.tiktok.c
            cache_time=1,
            is_personal=True,
        )
        return

    if not STORAGE_CHAT_ID:
        await q.answer(
            [article("nostorage", "Бот не настроен", "Не задан STORAGE_CHAT_ID",
            cache_time=1,
        )
        return

    url = clean(match.group(0))
    rid = hashlib.md5(url.encode()).hexdigest()

    try:
        file_id = await get_file_id(bot, url)
    except asyncio.TimeoutError:
        # не успели: качаем дальше в фоне, а юзеру говорим дописать символ
        await q.answer(
            [article(
                "wait",
                "⏳ Обрабатываю видео...",
                "Дай секунду и добавь пробел в конце ссылки, видос появится здесь
                url,
```

```
            )],
            cache_time=0,
            is_personal=True,
        )
        return
    except Exception as e:
        logging.exception("inline failed: %s", e)
        await q.answer(
            [article("err", "Не смог скачать 😕", "Фотопост, закрытое видео или T
            cache_time=0,
            is_personal=True,
        )
        return

    await q.answer(
        [InlineQueryResultCachedVideo(id=rid, video_file_id=file_id, title="Отпра
        cache_time=300,
        is_personal=True,
    )


async def main():
    if not TOKEN:
        raise SystemExit("Задай переменную окружения BOT_TOKEN")
    if not STORAGE_CHAT_ID:
        logging.warning("STORAGE_CHAT_ID не задан, inline-режим работать не будет
    bot = Bot(TOKEN)
    await dp.start_polling(bot)


if __name__ == "__main__":
    asyncio.run(main())
```